Control 2: Algorithms & Complexity 2526

This assignment consists on solving two optimization problems through efficient implementations in Python.

- The **Skill Tree Optimization** problem (`skill-tree-opt`) corresponds to Control 2 and is **mandatory**.
- The **DNA Compression** problem (`dna-comp`) corresponds to the Control 1 retake **and is optional**. The mark obtained in this exercise will be added to the mark of Control 1.

The assignment is intended to be solved by a **single student**.

You will need to use the `algodredd` command line tool to automatically evaluate your solutions over a given set of instances.

NOTE: Although you could develop the assignment in any OS, you will need to run **Linux** to evaluate your solutions using `algodredd` . For any issue, please contact me.

In this document you will find the following information:

- The project structure.
- A description for the problems that you will solve.
- How to use the `algodredd` tool to evaluate your solutions and automatically deliver the project.

The main goal of this project is to provide efficient implementations for the proposed problems, such that **they output correct solutions** and they are **comparatively equals or better** in terms of running time than the provided solutions.

WARNING! This assignment will be automatically evaluated using the `algodredd` tool. Make sure you follow all the steps described in this document before submitting your solution. Additionally, it is mandatory to deliver the assignment using the `algodredd deliver` command, not by uploading a zip file manually. **A project that cannot be automatically evaluated will receive a score of 0.**

Project Structure

This project has the following structure:

```
student/
├── assignment.json
├── group.json
├── skill-tree-opt
│   ├── instances
│   │   └── ...
│   ├── problem.json
│   ├── solution.py
│   ├── solutions
│   │   └── alg1.bin
│   ├── visualizer.py
│   └── validator.bin
└── dna-comp
    ├── instances
    │   └── ...
    ├── problem.json
    ├── solution.py
    ├── solutions
    │   └── alg1.bin
    └── validator.bin
```

The problems that you must solve are within the `student/skill-tree-opt` and `student/dna-comp` directories.

Problems Descriptions

Problem 1: The *Skill Tree Optimization* Problem

You are designing a character build for a role-playing game (RPG).

The game contains a **skill tree** where each skill:

- costs a certain number of skill points,
- provides bonuses to one or more character attributes,
- may require other skills to be unlocked first.

Additionally, some combinations of skills create **synergies** that provide extra bonuses.

Your goal is to select the best set of skills while respecting the skill-point budget.

Given a skill tree, a limited budget, attribute weights, and synergy bonuses, you must find the set of skills that maximizes the final score.

Rules

1. **Budget Constraint**: Each skill has a cost. The total cost of all selected skills must not exceed the available budget.
2. **Skill Dependencies**: Some skills require other skills to be selected first. For example, if `Precision` requires `Critical Strike`, you cannot select `Precision` unless `Critical Strike` is also selected.
3. **Attributes**: Skills modify character attributes such as damage, survivability, mana, mobility, critical chance. Each selected skill contributes to the total attribute values.
4. **Scoring Function**: Each attribute has an associated weight. The final score is computed as `weight_1 * total_attribute_1 + weight_2 * total_attribute_2 + ...`
5. **Synergies**: Some combinations of skills provide additional bonuses. A synergy becomes active only if **all required skills** are selected.

Input and Solution Format

Instances are located in the `student/skill-tree-opt/instances` directory. The instance parsing is already implemented for you in `student/skill-tree-opt/solution.py`. Similarly, the way solution is printed is also implemented for you (which corresponds to the set of skills selected).

You can use the `student/skill-tree-opt/visualizer.py` file to visualize the skill tree and the synergies for any of the provided instances.

Your Implementation

You are provided with the `student/skill-tree-opt/solution.py` file, where you must implement your solution. You already have all the necessary imports and a `main` function that reads the input and prints the output using the described formats, so **you should not change anything of this part**.

Instead, you should just implement the required algorithm function inside this file.

Provided Solutions and Validator

There are two provided solutions for this problem, located in the `student/skill-tree-opt/solutions` directory. These are Python implementations, but the code is obfuscated, so you should not be able to understand the algorithm used.

You can easily run any of the algorithms as shown:

```
./student/skill-tree-opt/solutions/alg1.bin $instance_file
```

This way, you can easily compare your solution with the provided solutions and check if they are correct.

Aside, you are provided with a `student/skill-tree-opt/validator.bin` file, which is a validator for the problem. You can use it to check if your solution is correct as shown:

```
# First, we need to store the output of the algorithm in a file
./student/skill-tree-opt/solutions/alg1.bin $instance_file > solution.json

# Then, we can use the validator to check if the solution is correct
./student/skill-tree-opt/validator.bin $instance_file solution.json
```

The validator will print the solution's value if it's correct. Otherwise, it will print an error message describing why the solution is not correct.

Scoring

We will execute your solution using a hold-out set of instances (which will be very similar to the ones provided). Therefore, you must try to deliver the most efficient algorithm possible.

The implemented algorithm must behave in the following way:

- It must be correct in all the tested instances (checked by the validator).
- It must return solutions that are equal or better to the ones provided by the reference algorithms.
- It must solve all the instances solved by the reference algorithms within the time limit.

Any algorithm that do not fulfill these requirements will receive a score of 0. For this exercise, the running time will not be considered (as long as you can solve all the instances within the time limit).

WARNING! This information is not represented in the summary table, you must check the execution of each individual instance to verify that it is correct. In the following example, you can see that the validator output is the same for all executions (with a small precision error), and is also equal to the one provided by the reference algorithm. Therefore, this instance is correctly solved:

```
[inst03] Status: AC
  Time: 0.028s, Memory: 13244KB
  Execution 1: Time: 0.026s, Memory: 13164KB, Status: AC
    Validator Output: 165.15
  Execution 2: Time: 0.028s, Memory: 13244KB, Status: AC
    Validator Output: 165.14999999999998
  Execution 3: Time: 0.032s, Memory: 13324KB, Status: AC
    Validator Output: 165.14999999999998
  Reference Comparison:
    alg1.bin: Time=0.087s, Mem=51768KB
      Validator Output: 165.14999999999998
    alg1.bin_mem_ratio: 0.2558337196723845
    alg1.bin_time_ratio: 0.31984601334088625
```

The score is computed in the following way:

1. Easy instances set: up to 3 points.
2. Medium instances set: up to 3 points.
3. Hard instances set: up to 3 points.
4. Bonus! Very hard instances set: 1 point if it solves at least one instance (notice that the provided algorithm is not able to find a solution for this set in the given time limit).

Problem 2: The *DNA Compression* Problem (Control 1 Retake Exercise)

DNA sequences are long strings composed of four nucleotides: A, C, G, and T.

Your task is to compress a DNA sequence using a simple dictionary-based encoding scheme. The goal is to minimize the total compressed size.

Compression Rules

You may encode the DNA sequence using two types of operations:

1. **Literal Encoding**: You may encode a single character directly. Encoding one character directly costs: `literal_cost`.
2. **Copy Encoding**: You may copy a substring that already appeared earlier in the sequence. A copy operation is represented by `(offset, length)` where `offset` is how far back the copied substring starts, and `length` is the number of copied characters. A copy operation costs: `copy_cost + length_cost * length` where `copy_cost` is the fixed overhead of using a copy, and `length_cost` is the cost per copied character. Note that you can only copy a substring that has at least length `min_match_length`.

Objective: Find a sequence of encoding operations that minimizes the total compression cost.

Input and SolutionFormat

Instances are located in the `student/dna-comp/instances` directory. The instance parsing is already implemented for you in `student/dna-comp/solution.py`.

For the output, you must build a list of dictionaries with the following format:

```
[
    {
        "type": "literal",
        "char": "A"
    },
    {
        "type": "literal",
        "char": "A"
    },
    {
        "type": "copy",
        "offset": 2,
        "length": 2
    }
]
```

This structure will already be printed in the required format.

Your Implementation

You are provided with the `student/dna-comp/solution.py` file, where you must implement your solution. As with the other problem, you already have all the

necessary imports and a `main` function that reads the input and prints the output using the described formats, so **you should not change anything of this part**.

Instead, you should implement the required algorithm that computes the optimal sequence of encoding operations.

Provided Solutions and Validator

There are two provided solutions for this problem, located in the `student/dna-comp/solutions` directory. As with the other problem, these are Python implementations, but the code is obfuscated, so you should not be able to understand the algorithm used.

You can easily run any of the algorithms as shown:

```
./student/dna-comp/solutions/alg1.bin $instance_file
```

This way, you can easily compare your solution with the provided solutions and check if they are correct.

Aside, you are provided with a `student/dna-comp/validator.bin` file, which is a validator for the *dna-comp* problem. You can use it to check if your solution is correct as shown:

```
# First, we need to store the output of the algorithm in a file
./student/dna-comp/solutions/alg1.bin $instance_file > solution.json

# Then, we can use the validator to check if the solution is correct
./student/dna-comp/validator.bin $instance_file solution.json
```

The validator will print the solution's value if it's correct. Otherwise, it will print an error message describing why the solution is not correct.

Scoring

We will execute your solution using a hold-out set of instances (which will be very similar to the ones provided). Therefore, you must try to deliver the most efficient algorithm possible.

The implemented algorithm must behave in the following way:

- It must be correct in all the tested instances (checked by the validator).
- It must return solutions that are equal or better to the ones provided by the reference algorithms.
- It must solve all the instances solved by the reference algorithms within the time limit.

Any algorithm that do not fulfill these requirements will receive a score of 0. For this exercise, the running time will not be considered (as long as you can solve all the instances within the time limit).

WARNING! This information is not represented in the summary table, you must check the execution of each individual instance to verify that it is correct. See the example provided for the *skill-tree-opt* problem for further details.

The score is computed in the following way:

1. Easy instances set: up to 4 points.
2. Medium instances set: up to 4 points.
3. Bonus! Hard instances set: 2 points if it solves at least one instance (notice that the provided algorithm is not able to find a solution for this set in the given time limit).

The `algodredd` Tool

You will receive within the assignment files a binary called `algodredd`. This tool will be used to evaluate and deliver your solutions.

As we stated before, `algodredd` **only works in a Linux system**. In case you are not using Linux, you can use a virtual machine, a Docker container, a *devcontainer* or WSL (Windows Subsystem for Linux) to run `algodredd`.

If you run `algodredd` without passing any parameters you will obtain a help message with the available commands and options:

```
Usage: algodredd <command> [arguments]
Commands:
  run     <solution.py> [set] [-p N] [-r N]  Run solution against public instances (or a specific set) with N parallel workers and N
  deliver <assignment_dir>             Package solutions into a delivery zip
  version
```

Evaluating Your Solutions

The `algodredd run` command allows you to run your solution against the public instances. Its parameters are:

- `<solution.py>`: The path to your solution file.
- `[set]`: The set of instances to run against. If not specified, it runs against all the public instances.
- `[-p N]`: The number of parallel workers to use. If not specified, it uses 1 worker.
- `[-r N]`: The number of repeats to run against each instance. If not specified, it uses 1 repeat.

For example, to run your *Skill Tree Optimization* solution against the *easy* set of instances with 4 parallel workers, you would use the following command:

```
./algodredd run student/skill-tree-opt/solution.py easy -p 4 -r 3
```

This should output the execution results, reporting the status for each instance (e.g., **AC** for Accepted, **WA** for Wrong Answer, **TLE** for Time Limit Exceeded, **RTE** for Runtime Error). Only **AC** means that your solution is correct and efficient enough to pass the time limits.

NOTE: `algodredd` uses a cache to store the execution results for the reference solutions (i.e., `alg1.bin`, `alg2.bin` ...) to speed up the evaluation process. You might find that the first execution of the `algodredd run` command is slower than the subsequent ones. This is expected behavior. In case you have any issues with this cache, you can remove the `.ref_cache.json` file, which is contained at each of the problems' folders.

Automatic Project Delivery

`algodredd` also provides an utility to pack your solutions into a zip file, which will be used to deliver your solutions to the grading system in the proper expected format, so you will not need to create it by yourself.

Before running this command, you must complete the `group.json` file with your group information. In particular, you must provide:

- "name": Your name and surnames.
- "id": Your student ID as it appears in the Virtual Campus.
- "dni": You DNI number (including the letter).

Once completed, you can run the `algodredd deliver` command. Its parameters are:

- `<assignment_dir>`: The path to the assignment directory.

This will create a `.zip` file in the `<assignment_dir>` folder, which you will have to submit through the Virtual Campus platform.